

NP-Complete Proof Chain

Compact Actual Proofs - Exam Ready

Proof Format to Use

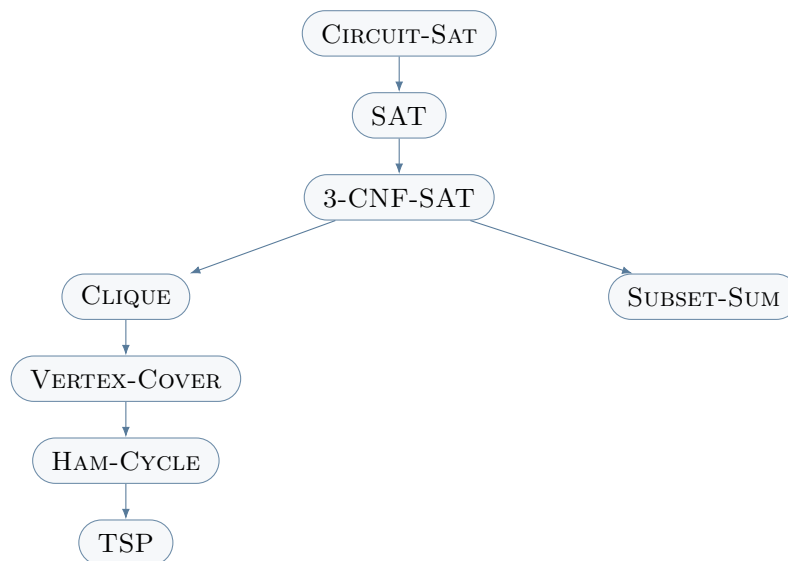
For every proof, write only these six parts:

In NP $\rightarrow$ Build $\rightarrow$ $\Rightarrow$ $\rightarrow$ $\Leftarrow$ $\rightarrow$ Time $\rightarrow$ Conclusion

Reduction Chain

CIRCUIT-SAT $\leq_p$ SAT $\leq_p$ 3-CNF-SAT $\leq_p$ CLIQUE $\leq_p$ VERTEX-COVER $\leq_p$ HAM-CYCLE $\leq_p$ TSP

3-CNF-SAT $\leq_p$ SUBSET-SUM.



1 Circuit-SAT is NP-Complete

Problem

Does a Boolean circuit have some input assignment that makes its output 1?

Actual Proof

In NP. Certificate = input bits. Evaluate each gate once. For g gates, verification is $O(g)$.

Build. Let L be any problem in NP with polynomial-time verifier $V(x, y)$. Fix input x . Build a Boolean circuit C_x whose free inputs are the certificate bits y and whose gates simulate the computation of $V(x, y)$.

$\Rightarrow$. If $x \in L$, some certificate y makes $V(x, y)$ accept. Using that y as circuit input makes $C_x = 1$.

$\Leftarrow$. If some input y makes $C_x = 1$, then the simulated verifier accepts (x, y) . Hence $x \in L$.

Time. V uses polynomially many steps, so the simulation circuit has polynomially many gates and is built in polynomial time.

Conclusion. Every problem in NP reduces to Circuit-SAT, and Circuit-SAT is in NP. Therefore

CIRCUIT-SAT is NP-complete.

2 SAT is NP-Complete

Problem

Does a Boolean formula have a satisfying truth assignment?

Reduction

$$\text{CIRCUIT-SAT} \leq_p \text{SAT}$$

Actual Proof

In NP. Certificate = truth assignment. Substitute values and evaluate the formula in polynomial time.

Build. From a circuit C , create one variable for every wire. For each gate add a constraint:

$$\begin{aligned} z = x \wedge y & : z \leftrightarrow (x \wedge y), \\ z = x \vee y & : z \leftrightarrow (x \vee y), \\ z = \neg x & : z \leftrightarrow \neg x. \end{aligned}$$

Conjoin all gate constraints and require the output variable o to be true:

$$F = (\text{all gate constraints}) \wedge o.$$

$\Rightarrow$. If C outputs 1 for some input, assign each wire-variable its actual circuit value. Every constraint is true and $o = 1$, so F is satisfiable.

$\Leftarrow$. If F is satisfiable, all wire values obey the gate rules and $o = 1$. Thus the original circuit has an input making output 1.

Time. One constant-size condition per gate, so $|F| = O(|C|)$.

Conclusion. Circuit-SAT $\leq_p$ SAT and SAT $\in$ NP, so

SAT is NP-complete.

3 3-CNF-SAT is NP-Complete

Problem

Does a CNF formula with exactly three literals per clause have a satisfying assignment?

Reduction

$$\text{SAT} \leq_p \text{3-CNF-SAT}$$

Actual Proof

In NP. Guess an assignment and check every clause.

Build. Introduce a helper variable for each subformula. Enforce each operation by clauses:

$$\begin{aligned} z = x \wedge y & : (\neg z \vee x)(\neg z \vee y)(z \vee \neg x \vee \neg y), \\ z = x \vee y & : (z \vee \neg x)(z \vee \neg y)(\neg z \vee x \vee y), \\ z = \neg x & : (z \vee x)(\neg z \vee \neg x). \end{aligned}$$

Force the root variable to be true. Convert shorter clauses to 3-literal clauses:

$$\begin{aligned} (a \vee b) & \rightsquigarrow (a \vee b \vee p)(a \vee b \vee \neg p), \\ (a) & \rightsquigarrow (a \vee p \vee q)(a \vee p \vee \neg q)(a \vee \neg p \vee q)(a \vee \neg p \vee \neg q). \end{aligned}$$

$\Rightarrow$. A satisfying assignment of the original formula extends to helper variables using the true values of their subformulas, so every new clause is satisfied.

$\Leftarrow$. Any satisfying assignment of the new formula forces each helper variable to represent the correct operation; since the root is true, the original formula is true.

Time. Each original operation creates only $O(1)$ variables/clauses.

Conclusion.

3-CNF-SAT is NP-complete.

4 Clique is NP-Complete

Problem

Does a graph contain k vertices that are all pairwise adjacent?

Reduction

3-CNF-SAT $\leq_p$ CLIQUE

For m clauses, ask for a clique of size m .

Actual Proof

In NP. Certificate = k vertices. Check all pairs for edges.

Build. Let $\phi = C_1 \wedge \dots \wedge C_m$. Create one vertex for each literal occurrence. Add an edge between two vertices iff:

- they come from different clauses, and
- they are not contradictory (x vs. $\neg x$).

Set $k = m$.

$\Rightarrow$. If ϕ is satisfiable, choose one true literal from each clause. The chosen literals never contradict, so all chosen vertices are connected. Hence there is an m -clique.

$\Leftarrow$. An m -clique can contain at most one vertex from each clause, so it contains exactly one from every clause. No two selected literals contradict. Make all selected literals true; then every clause is satisfied.

Time. $3m$ vertices and at most $O(m^2)$ edges.

Conclusion.

CLIQUE is NP-complete.

5 Vertex-Cover is NP-Complete

Problem

Can at most k vertices cover every edge of a graph?

Reduction

CLIQUE $\leq_p$ VERTEX-COVER

Map (G, k) to $(\overline{G}, |V| - k)$.

Actual Proof

In NP. Certificate = set C of at most k vertices. Check every edge has an endpoint in C .

Build. Construct the complement graph $\overline{G}$: two distinct vertices are adjacent in $\overline{G}$ exactly when they are not adjacent in G .

$\Rightarrow$. If S is a k -clique in G , then there is no edge of $\overline{G}$ inside S . Therefore every edge of $\overline{G}$ touches $V - S$, so $V - S$ is a vertex cover of size $|V| - k$.

$\Leftarrow$. If C is a vertex cover of $\overline{G}$ with $|C| \leq |V| - k$, then at least k vertices are outside C . No edge of $\overline{G}$ joins two outside vertices, so every pair of them is adjacent in G . They contain a clique of size k .

Time. Constructing $\overline{G}$ takes $O(|V|^2)$.

Conclusion.

VERTEX-COVER is NP-complete.

6 Hamiltonian Cycle is NP-Complete

Problem

Does a graph contain a cycle visiting every vertex exactly once?

Reduction

$\text{VERTEX-COVER} \leq_p \text{HAM-CYCLE}$

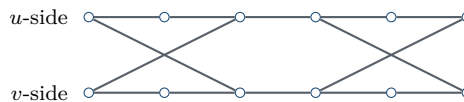
Replace each original edge by the standard edge gadget; connect gadget sides into one chain per original vertex; add k selector vertices.

In NP

Certificate = an ordering of all vertices. Check uniqueness, every consecutive edge, and the final edge back to the start.

Edge Gadget

For each original edge $\{u, v\}$ create a 12-vertex gadget with a u -side and a v -side:



Only Gadget Fact Needed

A Hamiltonian traversal of this gadget can represent exactly these useful cases:

u chosen v chosen u and v both chosen.

Thus the gadget can be fully visited iff at least one endpoint of the original edge is selected.

Actual Correctness Proof

Build. Join all u -sides of gadgets incident to each original vertex u into one long u -chain. Add selector vertices $s_1, \dots, s_k$ and connect each selector to the two ends of every vertex-chain.

$\Rightarrow$. Let C be a vertex cover with $|C| \leq k$ (pad to exactly k if needed). Use selectors to connect the chains of vertices in C . For every original edge $\{u, v\}$, at least one endpoint is in C , so use the gadget traversal corresponding to u , v , or both. Hence every gadget vertex is visited exactly once, producing a Hamiltonian cycle.

$\Leftarrow$. Given a Hamiltonian cycle, remove the k selector vertices. The cycle splits into at most k paths, each following one original vertex-chain. Call those original vertices selected. Every edge gadget must be fully visited, and by the gadget fact this requires at least one endpoint-chain of that edge to be selected. Therefore the selected vertices cover every original edge, so they form a vertex cover of size at most k .

Time. Each edge gives 12 vertices; all chain and selector connections are polynomial in $|V| + |E|$.

Conclusion.

HAM-CYCLE is NP-complete.

7 Traveling-Salesman Problem is NP-Complete

Problem

Is there a tour visiting every vertex once, returning to the start, with cost at most B ?

Reduction

HAM-CYCLE $\leq_p$ TSP

For $n = |V|$, make a complete graph with cost 1 on original edges, cost 2 on missing edges, and set $B = n$.

Actual Proof

In NP. Certificate = tour order. Check all vertices once and sum the n edge costs.

Build.

$$c(u, v) = \begin{cases} 1, & \{u, v\} \in E, \\ 2, & \{u, v\} \notin E. \end{cases} \quad B = n.$$

$\Rightarrow$. A Hamiltonian cycle in G uses n original edges, each of cost 1, so the same tour in G' has cost n .

$\Leftarrow$. Any TSP tour uses exactly n edges. Since every edge costs at least 1, total cost $\leq n$ is possible only if every used edge costs 1. Therefore all tour edges were original edges of G , so the tour is a Hamiltonian cycle.

Time. Assigning costs to all $O(n^2)$ pairs is polynomial.

Conclusion.

TSP is NP-complete.

8 Subset-Sum is NP-Complete

Problem

Given positive integers and target t , is there a subset whose sum is exactly t ?

Reduction

3-CNF-SAT $\leq_p$ SUBSET-SUM

Use decimal columns: one per variable and one per clause.

Actual Proof

In NP. Certificate = chosen numbers. Add them and compare with t .

Build. For variables $x_1, \dots, x_n$ and clauses $C_1, \dots, C_m$, use $n + m$ decimal columns.

- For each x_i , create T_i and F_i . Both have digit 1 in variable column i .
- In clause column j , T_i has 1 if $x_i \in C_j$; F_i has 1 if $\neg x_i \in C_j$.
- For each clause add two slack numbers with digits 1 and 2 in that clause column.
- Target digits: variable columns = 1; clause columns = 4.

No carry occurs because every column sum is below 10.

$\Rightarrow$. From a satisfying assignment choose exactly one of T_i, F_i per variable. Each variable column

becomes 1. A satisfied clause receives 1, 2, or 3 from true literals; add slack to reach 4:

$$1 \rightarrow 1 + 2, \quad 2 \rightarrow 2, \quad 3 \rightarrow 1.$$

So the chosen numbers sum to the target.

$\Leftarrow$. If a subset sums to the target, each variable column equals 1, so exactly one of T_i, F_i is chosen; this defines a truth assignment. In a clause column, slack contributes at most 3 but the target is 4, so at least one chosen variable number contributes 1. Hence that clause has a true literal. Therefore all clauses are satisfied.

Time. We create $2n + 2m$ numbers, each with $n + m$ digits, so the construction is polynomial.

Conclusion.

SUBSET-SUM is NP-complete.